Word Embeddings Based on Graphical Relationship Between Words and Their Definitions

Nicholas Smith

LING 581 - Fall 2023

Abstract

Context-based algorithms for generating word embeddings are known to capture the inherent biases within the text the embeddings are trained on. In this paper we attempt a different approach for generating word embeddings using a dictionary (WordNet) and a graph of which words define which other words. Embeddings are generated by using the Node2Vec algorithm on both an unweighted and a weighted version of the graph. Results are compared with pre-trained Word2Vec embeddings from Google and inspected for gender bias. The embeddings generated appear to capture some semantic relationships between words in the same way Word2Vec algorithms are known to. They group related words together and appear to be relatively unbiased.

1 Introduction

Common practice for generating word embeddings for use in various machine learning and Natural Language Processing tasks is to train the embeddings for each word based on the context the word appears in. Two common algorithms in the Word2Vec package are Continuous Bag of Words and Skip-Gram (Mikolov et al., 2013). Embeddings trained using these algorithms are known to capture semantic relationships between words, for example, the embedding for ’man’ is related to ’king’ in a similar way to how the embedding for ’woman’ is related to ’queen’. However, also captured in these relationships is the inherent bias that exists in the corpus the embeddings are trained on. For example, embeddings trained on the internet might relate ’man’ to ’computer programmer’ in the same way they relate ’woman’ to ’homemaker’. They also may exhibit racial or religious bias that can be perpetuated into whatever model may use the embeddings (Bolukbasi et al., 2016).

In this paper, we explored a different approach for generating word embeddings that does not involve using the context words, but rather their dictionary definitions, to capture the meaning and relationships between words. The approach was to create a graph of which words define which other words, and let the structure inherent within the graph guide the creation of node (representing words) embeddings using the Node2Vec algorithm (Grover and Leskovec, 2016).

Dictionary definitions tend to be fairly unbiased and have high signal in regard to the actual semantic meaning of words. The definition for "Computer Programmer" in WordNet (University, 2010), for example, is "a person who designs and writes and tests computer programs." There is nothing there linking being a computer programmer to being a man. On the other hand, the WordNet definition for king is "a male sovereign; ruler of a kingdom" while the WordNet definition for queen is "a female sovereign ruler." These words have similar definitions, the main degree of separation being, of course, the gender, which is, of course, the main difference between the meaning of words themselves.

The hope was that embeddings created using the structure of the graph created using definitions would continue to capture these fundamental relationships between words while filtering out the bias that only has to do with how they are used.

We employed three different attempts at creating word embeddings from our graph, one on an unweighted version of the graph and two on a weighted version of the graph. These methods are discussed later in the paper. For comparison we used a set of pre-trained embeddings from Google (Google, 2013) that were generated with a context-based approach.

We evaluated the results of our model using a set of analogies found in a public repository (Léonard, 2013) and by generating a plot to check the gender bias inherent in our model.

2 Related Work

Much other work has been done on eliminating bias in word embeddings. Some methods include trying to remove the bias after the embeddings have trained, for example, there are methods of calculating the bias and subtracting it off (Bolukbasi et al., 2016). These methods however, alter the embeddings themselves and the relationships they may have with other words. Other methods include data augmentation when training the embeddings (Zhao et al., 2018). Our method is unique in that it creates the embeddings using an entirely different approach that is not context-based at all. It also does not require a large number of examples or augmented data to generate the embeddings, but still has enough information to represent essentially every word in a language, without representing unnecessary information or bias.

3 Methods

Our method for generating our embeddings involved two parts. First, building the graph, and next, running the Node2Vec algorithm to generate the embeddings.

We started building our graph of words by using Princeton University's WordNet model (University, 2010) as our dictionary and source of words. We chose WordNet because it is in the NLTK library and could be imported relatively easily. It also simplified our model because it only has information on nouns, verbs, adjectives, and adverbs, words with specific meanings rather than words used for grammatical purposes. We could link words to words that actually contributed to their meaning, rather than having ten connections to articles and conjunctions.

A primary challenge we faced involved disambiguating word senses within the graph. To prevent the conflation of distinct lexical categories – such as the noun 'watch' versus the verb 'watch' – we required nodes to encode both the lemma and its Part of Speech (POS) tag. While WordNet indexes the base forms of words, it lacks inherent support for inflected surface forms (e.g., pluralization or present participles). Our approach aimed to map these various morphological variants back to their respective base-form definitions while maintaining a structural distinction between the base lemma and its derived forms.

3.1 Generating Nodes With POS Tags

We achieved POS-tagged nodes by leveraging the NLTK library's default POS-tagger. We started by creating a mapping from the standard list of POS tags used by NLTK to the list of POS tags included in WordNet – for example NNS would be mapped to "n" – but we also included other attributes of a particular POS tag when we created an actual node representing the word. The node for dog, for example, would be ('dog', 'n', 'singular', 'common'), representing the NN tag for a singular, common noun. The node for dogs, on the other hand, would be ('dogs', 'n', 'plural', 'common'). The node for digging (tagged VBG) would be ('digging', 'v', 'present', 'participle'), and so forth.

To get additional contexts for words and a larger vocabulary, we expanded our corpus to include a few external ebooks, namely a version of Webster's English Dictionary (Various, 2009), and a few others. This was primarily for the purpose of getting all of the possible POS tags for each word. We also POS-tagged each of the definitions in WordNet, each example sentence they had, and each lemma in each synonym set.

WordNet has a method for determining if two words have the same base, so even though "dogs" is technically not in the model, if you pass it into the wn.synset() function, it will still return the synonym set and definition for "dog". It also allows filtering by part of speech. This way, we were able to pass all of our nodes through the model and check which ones actually returned a definition for each part of speech. We primarily kept only nodes that had at least one definition in WordNet.

That said, some words were POS-tagged incorrectly and needed fixing. For example, easygoing is an adjective, but it was consistently POS-tagged as a verb. To solve this, we included an additional check. If a word did not have a definition for its predicted POS tag in WordNet, but the word itself was in WordNet, we still included it. In the case where WordNet only had a single POS tag for the word, we assumed that to be the correct POS tag and changed it. In the case where WordNet had multiple POS tags for the word, we did not make any assumptions, but simply included the word as an additional node with UNK_POS tag. This was important for when we went to POS tag the definitions and actually create the graph linking words together. If a word in a definition came back with an unknown POS tag, it could still be linked to the

UNK_POS node instead of not being included at all.

Once we had all of our nodes, it was straightforward to also POS tag each definition, filter to find the words with existing nodes in the graph, and thus create edges connecting words to the words in their definitions. However, to distinguish between words with the same definition but of different forms, we also included meta nodes in the graph that represented information like "plural", "singular", "participle", "present", etc. The idea was that the different forms of words would have the same definition, but be separated only in which meta node they were linked to. This, we hoped, would result in the same relationship existing between, for example, "run" and "running", as between "sit" and "sitting" when we actually generated the embeddings.

3.2 Node2Vec Algorithm

Node2Vec ((Grover and Leskovec, 2016) generates node embeddings of a graph by generating random walks through the graph to get an idea of the neighborhood of each node. It then uses the walks as the "context" for each node and generates embeddings in the same way as Word2Vec, only, rather than predicting which other words appear in the context of a word to generate embeddings, it is predicting which nodes are neighbors of a node. The tricky part of using Node2Vec to generate our word embeddings was balancing capturing the structure of the graph without relating words that are totally unrelated.

The basic form of the algorithm has two parameters, p and q, with q determining the likelihood of traveling deeper and traversing to a new neighbor in the graph, and p controlling the likelihood of returning to the previously visited node. This algorithm doesn't take into account edge weight, and so it wasn't suitable for our weighted version of our graph, as discussed in the next section; however it did work for our unweighted graph. In the unweighted case, we used a p and q of 1 and .25 respectively. This is because, while words are generally related to words in their definition, by the time it gets to the definition of a word in the definition of a word in the definition of a word, the chance of it being related to the first word is much smaller. We also didn't want nodes to be related to meta nodes that were not their own, and that risk increases the deeper the traversal on the graph.

Other parameters we used for comparison included 10 random walks per node with walks of length 30. These were different from the parameters we used on our weighted graph because we were experimenting with going deeper and simply letting the algorithm see more of the graph structure overall.

3.3 Weighting the Edges

Once we had our base graph, we wanted to also incorporate the degree to which each word gave meaning to the others, and have that influence our random walks. The thought was that if a word is in the definition of a lot of different words, then perhaps it doesn't contribute all that much to the meaning of any particular one of them. On the other hand, if a word is in the definition of only one word, then it is likely to contribute a lot to that word's definition.

We captured this idea by weighting each edge with the reciprocal of the target node's in-degree. These weights were then normalized by the sum of all outgoing edges from the source node, and used to establish the transition probabilities for a random walk through the graph.

As an example, we see in Figure 1 that Node A has an in-degree of 3, so each edge coming into Node A gets an initial weight of $\frac{1}{3}$. However, node E only has an in-degree of 1, so its initial edge weight remains at 1.

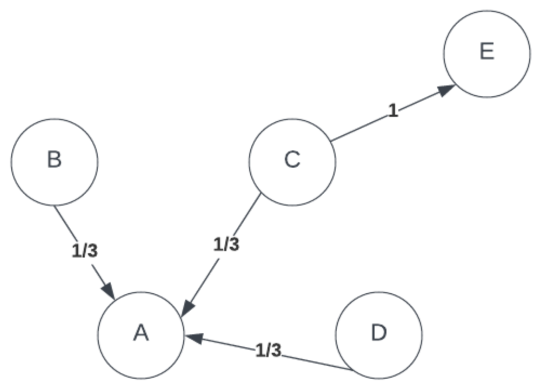

Figure 1: Un-normalized probabilities based on the in-degree of the node to which an edge is pointing.

After normalizing the weight of each edge to form probabilities, we get the graph in Figure 2. Using this method, however, generally puts too small of a probability on each of the meta nodes, each of which had hundreds or thousands of edges. Thus, in addition to using the reciprocal of the in-degree of a node, we also added a constant value to each edge weight before normalizing them to probabilities to ensure that no one edge had a probability that was too small. The value we chose in this case was 0.05.

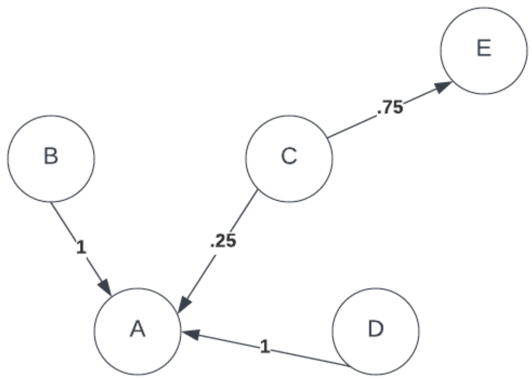

Figure 2: Normalized probabilities based on the weight of all edges going out of each node.

3.4 Modified Node2Vec

Because the standard Node2Vec algorithm is meant for an unweighted graph, we had to modify the algorithm for use on our weighted graph. In our case, rather than using q and p to represent the likelihood of traversing to a new node or returning to a previously visited one, we used p as the raw, unnormalized probability of returning to the previous node in the random walk and q as the value by which to scale the weight of each out-going edge (as calculated in the previous section) to get the raw likelihood of traversing to a new node. This means the true probability of traversing to the previous node in the path from node i was:

$$\frac{p}{p + q\sum_{j=1}^{n_{out}} w_{ij}} \tag{1}$$

While the probability of traversing from node i to node j that was a neighbor of node i was:

$$\frac{q\sum_{j=1}^{n_{out}} w_{ij}}{p + q\sum_{j=1}^{n_{out}} w_{ij}} \tag{2}$$

Where w_{ij} is the weight from node i to node j as calculated previously.

We also did not allow traversal from a node to its related meta nodes unless the node was the starting node in the path.

Once we generated our random walks (30 per node to ensure sufficient coverage), we ran the Word2Vec algorithm on our paths, once with a window size of 3, and then again with a window size of 5. Part of the reason for the small window sizes was, again, that we didn't want the embeddings for our nodes getting pushed too close to nodes that were too far away in the graph and unrelated.

Once the algorithms were complete, however, we were left with three sets of embeddings, one from the unweighted graph and two from the weighted graph with different window sizes.

4 Evaluation

4.1 Analogies

To evaluate our model, we used a set of analogies available in a public repository (Léonard, 2013) to test the semantic relationships between words and compared them to a pre-trained model from Google, the results of which were not too exciting.

The analogies came in twelve categories, which can be seen in Figures 3 and 4 in the Appendix. We tested on Google's embeddings by simply plugging in the words (word1 is to word2 as word3 is to word4) directly into the formula:

$$word1 - word2 + word3 = result \tag{3}$$

And then finding the embedding in the model that was nearest to the result. If the resulting embedding matched what was expected, it was counted as correct, and the total percentage correct was calculated for each category. We calculated two other versions of the accuracy as well, one counting it correct if the desired word appeared in the top 5 nearest neighbors to the result, and another counting it correct if the desired word appeared in the top 10 nearest.

With our models, we also had to account for part of speech, so we assumed the part of speech of word1 and word3 were the same, and that the parts of speech in word2 and word4 were the same. We then used the same formula and only counted it as correct if the word of the correct part of speech was in the top n results.

None of the models performed phenomenally well on these tests. However, Google's pretrained embeddings did significantly better in capturing the relationships between both singular words and their plural forms, and between the base forms of verbs and their present-participle forms. In the other categories, our weighted models didn't perform too much worse. In fact, for family relationship (e.g.

man is to father as woman is to mother) and nationality (e.g. Dutch is to Netherlands as English is to England) analogies, our weighted models performed better than Google's did.

Our unweighted model performed the worst of all of them; however, we would guess because it traversed too far away into nodes that were unrelated to the source node.

Overall, the degree to which our weighted model captured the semantic relationships is comparable to that of the pre-trained Google embeddings.

4.2 Evaluating for Bias

Where our models really did seem to shine was in yielding unbiased results. We had ChatGPT generate a list of occupations, sports, and instruments that were stereotypically filled or played by men or women. (For example, 'engineer' and 'basketball' would fall under the 'man' category, while 'homemaker' or 'flute' would fall under 'woman'). We found the embeddings for each of these words in each of our models and compared them using cosine similarity to the embeddings for 'man' and 'woman', then plotted each word on a grid with the cosine similarity to man on the x-axis and the cosine similarity to woman on the y-axis.

The resulting plots for each model are included in Figure 5 in the Appendix. Orange dots in the plots represent words that were labeled as being stereotypically women's activities, and blue dots are labeled as being stereotypically men's activities. We can see in the plot for Google's model that there is, for the most part, a very clear line of separation. Almost all of the orange dots fall closer to the embedding for 'woman' than they do to the embedding for 'man', showing very clearly the inherent bias that exists and comes from word typical contexts. In each of our models, however, the dots are more randomly spread around the line $y = x$, indicating no real relationship between the perceived gender of an activity or career and the embedding it lies closer to.

5 Future Work

Of course, there is likely more work that can be done to further improve our embeddings. We could certainly experiment with more parameters in Node2Vec to see if we can get better word representations. We can also experiment with adding additional meta nodes, such as a node for 'NOT', to see if we can more effectively capture relationships between opposites.

We could also certainly test our models for other types of biases, including racial or religious bias. Here we only covered a small subset of gender bias. The results look promising, though.

Lastly, a logical next step would be to test our embeddings in a language model to see how they perform. It may be that having words related only to words they are defined by is not as helpful in trying to predict the next word in a sentence as using the context they actually appear in, and perhaps context-based models would perform better there. However, that calls for additional testing.

6 Conclusion

In this paper we explored an alternative approach for generating word embeddings using a graphical approach, defining the relationships between words using their dictionary definitions, and then using a variation on the Node2Vec Algorithm to generate the actual embeddings. The resulting embeddings appear comparable to Google's pre-trained Word2Vec embeddings when it comes to capturing semantic relationships between words. However, generating embeddings using this method seems like a promising way to eliminate gender, and perhaps other types of harmful bias, and make them safer for the language models that use them.

References

Tolga Bolukbasi, Kai-Wei Chang, James Zou, Venkatesh Saligrama, and Adam Kalai. 2016. Man is to computer programmer as woman is to homemaker? debiasing word embeddings.

Google. 2013. word2vec-google-news-300. Word embeddings. December 18, 2023.

Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks.

Nicholas Léonard. 2013. questions-words.txt. https://github.com/nicholas-leonard/word2vec/blob/master/questions-words.txt. Accessed: December 18, 2023.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space.

Princeton University. 2010. Wordnet: A lexical database for the english language. https://wordnet.princeton.edu/. Accessed: [Insert Date].

Various. 2009. Webster's unabridged dictionary. https://www.gutenberg.org/ebooks/29765.

Jieyu Zhao, Tianlu Wang, Mark Yatskar, Vicente Ordonez, and Kai-Wei Chang. 2018. Gender bias in coreference resolution: Evaluation and debiasing methods.

A Appendix

google_word2vec

	adj-adverb	capitals_of_countries	cities_in_states	comparative	currency	family_analogies	nationalities	opposites	plural-verbs	plural	present-participle	superlative
top1	0.000000	0.000000	0.000000	0.000000	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0
top5	0.011111	0.004113	0.001349	0.003096	0.0	0.045455	0.000866	0.027981	0.176877	0.572638	0.341087	0.0
top10	0.053846	0.010135	0.005057	0.010320	0.0	0.172727	0.007359	0.055961	0.285820	0.694094	0.566051	0.0

weighted_window3

	adj-adverb	capitals_of_countries	cities_in_states	comparative	currency	family_analogies	nationalities	opposites	plural-verbs	plural	present-participle	superlative
top1	0.000000	0.000294	0.000000	0.000000	0.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
top5	0.054701	0.012779	0.000000	0.139061	0.0	0.108485	0.022727	0.002433	0.117342	0.263780	0.058594	0.111545
top10	0.106838	0.024530	0.001011	0.231682	0.0	0.217576	0.058874	0.003650	0.248518	0.444685	0.128551	0.229601

Figure 3: Analogy accuracies for Google Model and Weighted Model...

weighted_window5

	adj-adverb	capitals_of_countries	cities_in_states	comparative	currency	family_analogies	nationalities	opposites	plural-verbs	plural	present-participle	superlative
top1	0.000000	0.000147	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
top5	0.073504	0.019830	0.000674	0.100877	0.000000	0.108485	0.039610	0.000000	0.106966	0.239567	0.063565	0.075521
top10	0.140171	0.034224	0.003372	0.208720	0.002269	0.216364	0.083333	0.001217	0.215168	0.421850	0.129084	0.181424

unweighted

	adj-adverb	capitals_of_countries	cities_in_states	comparative	currency	family_analogies	nationalities	opposites	plural-verbs	plural	present-participle	superlative
top1	0.011966	0.002497	0.000000	0.000774	0.000000	0.000606	0.001732	0.001217	0.003706	0.002165	0.000533	0.000000
top5	0.071795	0.038484	0.001349	0.068369	0.043116	0.052727	0.134848	0.001217	0.053113	0.106496	0.046875	0.057726
top10	0.087179	0.065511	0.004383	0.123581	0.075643	0.093333	0.203247	0.002433	0.097579	0.193110	0.083984	0.093750

Figure 4: Analogy accuracies for Unweighted Model and Weighted Model...

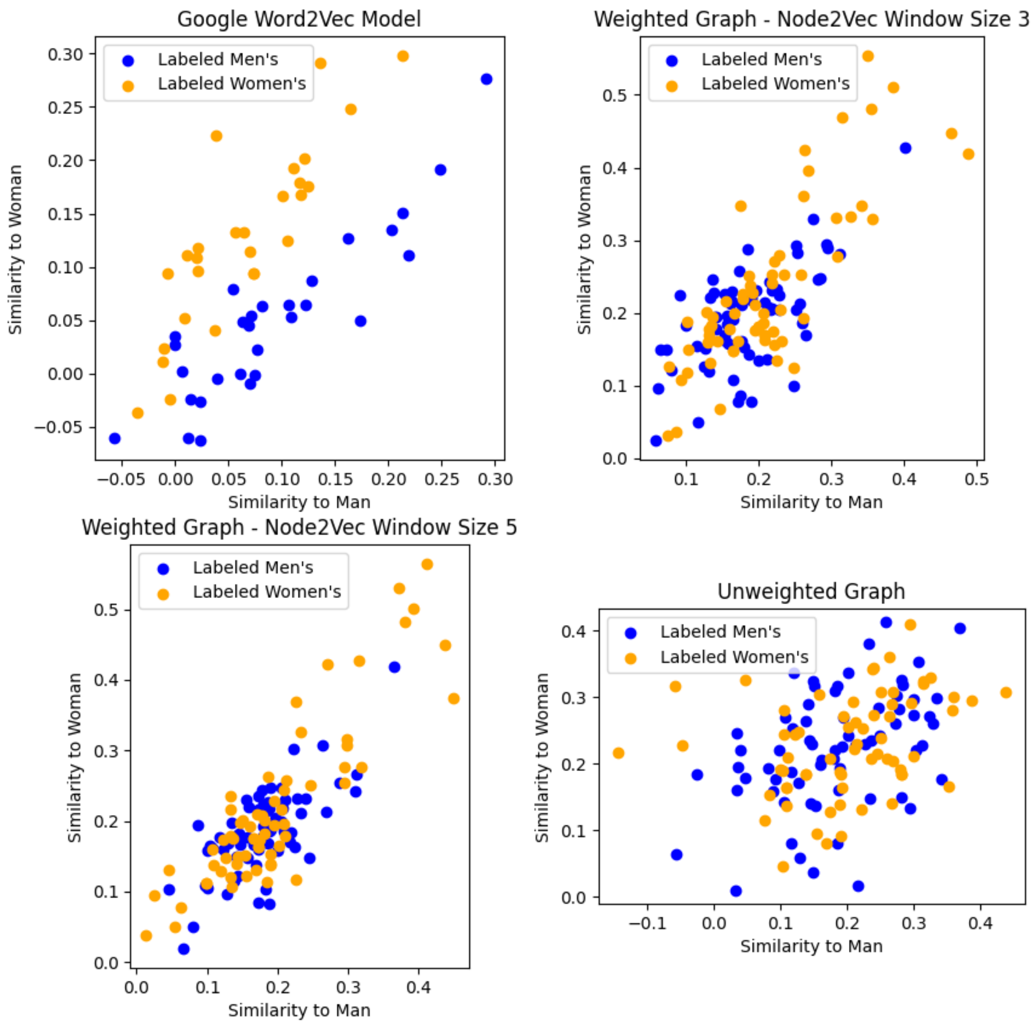

Figure 5: Google's Vectors show a distinctive split where words that are stereotyped as being women's jobs or activities are clearly closer to the woman vector and activities and words that are typically stereotyped to be men's jobs are closer to man. Our embeddings do not exhibit this bias.